

Simulation multi-agents : de la théorie cinétique des gaz à la modélisation de la condensation

Jessen Page et Florian Desmons

Computational physics 2

Filière : Informatique et systèmes de communication ISC

La simulation multi-agents est une approche informatique qui permet de modéliser et d'étudier les interactions entre les molécules au sein d'un système physique. Dans ce contexte, chaque molécule est représentée comme un agent autonome, doté de propriétés et de règles comportementales lui permettant de réagir à son environnement. Les agents (molécules) interagissent dynamiquement les uns avec les autres et avec leur environnement. Ainsi, dans ce grand laboratoire, vous allez implémenter votre modèle de simulation multi-agents d'interactions des molécules dans un système fermé.

Il est important de noter que ce code constitue un **projet évolutif**. Vous commencerez par implémenter une version simple de votre code puis vous le complèterez étape après étape au cours du semestre, au fur et à mesure de la découverte de nouveaux concepts physiques. Votre objectif est donc non seulement d'implémenter un code **modulaire et fonctionnel** mais aussi de comprendre comment un comportement complexe peut émerger de **règles simples** appliquées à un ensemble d'agents.

L'utilisation de l'IA est autorisée pour vous aider à comprendre les concepts ou générer une partie de votre code. Par contre, si vous l'utilisez pour générer du code, **assurez-vous de vraiment comprendre** ce qu'il fait. Pendant l'évaluation, **vous devrez être capables d'expliquer vos choix et votre implémentation**.

1 La base du code : la classe Molecule

La figure 1 est une schématisation d'une molécule et de ses caractéristiques dans ce laboratoire. Chaque molécule est décrite par ses composantes physiques suivantes :

- la position dans un référentiel galiléen de son centre de masse définie par $\vec{r}(t) = (x(t), y(t), z(t))$
- sa vitesse définie par $\frac{d}{dt}\vec{r}(t) = \vec{v}(t) = (v_x(t), v_y(t), v_z(t))$
- sa masse m
- le rayon ρ de la sphère modélisant la forme de la molécule
- sa formule chimique

Dans l'ensemble de ce laboratoire, les équations et algorithmes à implémenter utiliseront ces notations.

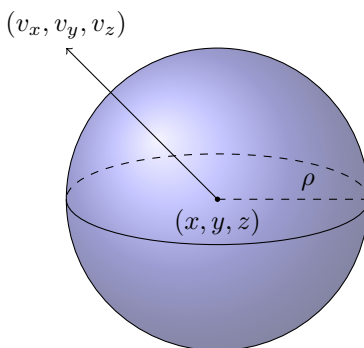


Figure 1: Schéma de la représentation d'une molécule et de ses composantes.

Exercice 1.1 :

Développez une classe nommée “Molecule” qui représente le modèle atomique présenté précédemment. Cette classe doit contenir les attributs suivants pour décrire une molécule :

- la masse (kg)
- le rayon de la sphère (m)
- la formule chimique
- la position spatiale du centre de la molécule (m)
- la vitesse de la molécule (m/s)

L'implémentation de cette classe vous permettra de modéliser le mouvement de chaque molécule comme étant un objet/agent.

2 Différents types de molécules

Exercice 2.1 :

Trouvez les valeurs des attributs de la classe “Molecule” pour les molécules suivantes : He , Ne , N_2 , O_2 .

Exercice 2.2 :

Quelles sont les différences entre ces molécules ? Est-ce que la théorie cinétique des gaz fonctionne pour toutes ces molécules ?

3 Le déplacement d'une molécule

Maintenant que vous avez réussi à créer la classe “Molecule”, vous allez mettre en mouvement celle-ci. Il est possible de simplifier sa dynamique sous la forme d'un mouvement de projectile sans interaction avec son environnement.

Question 3.1 :

En utilisant la seconde loi de Newton, écrivez les équations gouvernant le mouvement de ces molécules.

Question 3.2 :

Utilisez les différences finies sur le système d'équation obtenu à la question précédente.

Question 3.3 :

Implémentez ce calcul dans la méthode *computeNextPosition* calculant la position de la molécule au pas de temps d'après. Pour cette méthode, il vous est fortement conseillé de mettre en argument l'instance de la molécule puis de changer sa position directement dans la fonction.

Question 3.4 :

Afin de vérifier et valider la nouvelle fonction implémentée, vous allez instancier une des molécules de l'exercice 2.1 et visualisez sa trajectoire au cours du temps. Quels position, vitesse, pas de temps et temps final avez-vous choisi et pourquoi ?

4 Interaction entre deux molécules

Lorsqu'un système contient plusieurs molécules en mouvement, deux molécules peuvent s'approcher suffisamment l'une de l'autre et interagir entre elles.

Question 4.1 :

Décrivez le type d'interaction qui peut se produire lorsque deux molécules s'approchent suffisamment l'une de l'autre. Expliquez également dans quelles conditions ce modèle d'interaction est pertinent et quelles hypothèses permettent de l'utiliser dans ce contexte.

Question 4.2 :

Avant d'implémenter le modèle de collision, proposez au minimum 4 scénarios de test permettant de vérifier que votre code fonctionne correctement. Vos cas de test doivent notamment permettre de contrôler : la conservation de la quantité de mouvement, la conservation de l'énergie cinétique et l'influence de l'angle de contact entre les deux molécules. Décrivez clairement chaque scénario (conditions initiales).

Question 4.3 :

Rédigez une fonction qui détecte si deux molécules sont en interaction. Cette fonction doit renvoyer true lorsque la distance entre leurs centres est inférieure ou égale à la somme de leurs rayons et false dans le cas contraire.

Question 4.4 :

Implémentez l'algorithme étudié en cours avec Jessen permettant de mettre à jour l'état des molécules après une collision. À l'aide des cas de test définis à la question 4.2, vérifiez que votre implémentation respecte les propriétés physiques attendues.

5 Le domaine de simulation

Le domaine représente l'espace dans lequel vos molécules évoluent et interagissent. Dans ce laboratoire, cet espace est modélisé comme un pavé droit (un cuboïd), de dimensions $L_x \times L_y \times L_z$ centré autour de la position $(0, 0, 0)$. La figure 2 illustre schématiquement ce domaine.

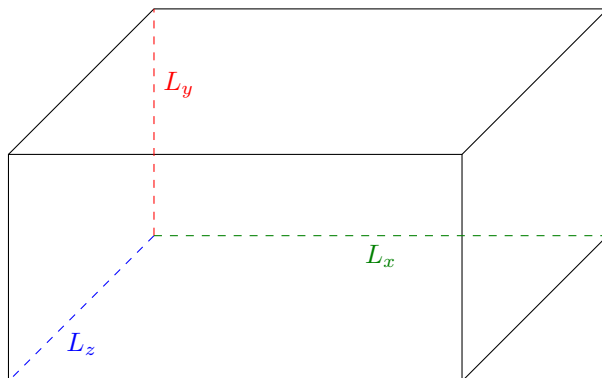


Figure 2: Schéma de la représentation d'un cuboïd et de ses caractéristiques.

Question 5.1 :

Développez une nouvelle classe nommée “Domain” qui représente le volume dans lequel se trouvent les molécules. Cette classe doit contenir les attributs suivants :

- la longueur de l'axe x : L_x (m)
- la longueur de l'axe y : L_y (m)
- la longueur de l'axe z : L_z (m)

Question 5.2 :

Ajoutez une fonction permettant de calculer automatiquement son volume.

Question 5.3 :

Après avoir proposé une vérification de votre classe et de votre méthode de calcul du volume, implémentez cette vérification pour vous assurer de leur bon fonctionnement.

6 Condition de bord/limite : réflexion spéculaire

Après avoir effectué un pas de temps Δt , il est possible qu'une molécule sorte du domaine de simulation. Dans la réalité physique, elle aurait heurté une paroi et rebondi. Pour simplifier la modélisation, nous considérons une réflexion spéculaire : la molécule rebondit comme un rayon lumineux sur un miroir.

L'idée est la suivante : si après un pas de temps, une molécule se trouve en dehors du domaine alors elle a traversé une paroi. Pour chaque axe où cette sortie se produit :

- on corrige la position en la “réfléchissant” par rapport au mur
- on inverse la composante de vitesse correspondante

La figure 3 est une représentation en deux dimensions de cette réflexion. Un atome (représenté par un disque bleu) est sorti du domaine (zone blanche) et, après avoir avancé d'un pas de temps, se situe désormais en dehors de l'espace possible (zone grise). Sur le schéma, il a traversé le bord droit du domaine d'une distance d et possède une vitesse (v_x, v_y) . Pour replacer correctement l'atome dans le domaine, il est repositionné sur le bord du domaine et subit une translation l'écartant d'une distance d de celui-ci. Ensuite, sa vitesse est modifiée à cause de l'impact de l'atome sur le bord changeant le signe de la vitesse de l'axe x transformant sa vitesse en $(-v_x, v_y)$.

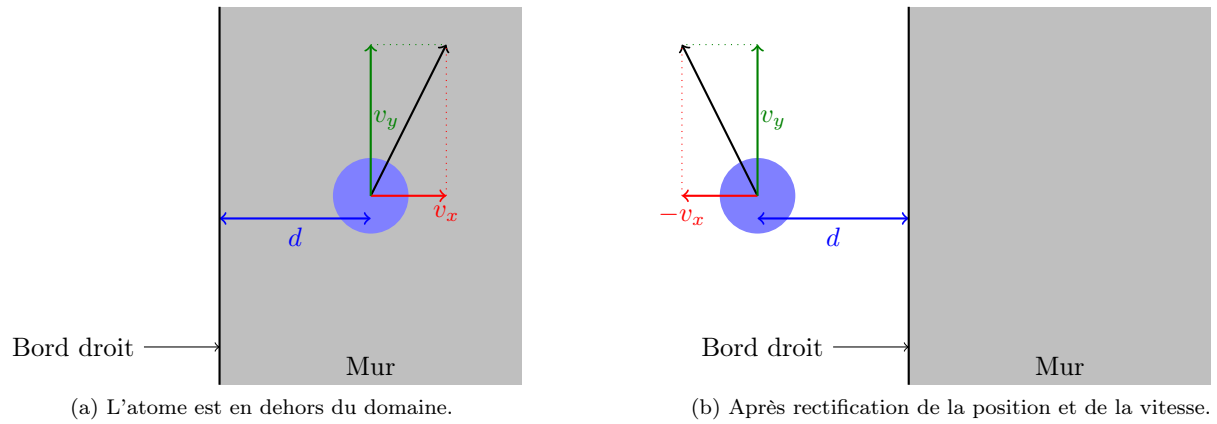


Figure 3: Schéma en deux dimensions de la condition de bord .

Question 6.1 :

Implémentez une fonction qui corrige la **position** et la **vitesse** d'une molécule lorsqu'elle se retrouve en dehors du domaine. Cette fonction doit vérifier chaque axe séparément et appliquer la réflexion si nécessaire.

Question 6.2 :

Proposez puis implémentez au minimum 8 scénarios de test permettant de vérifier que votre code fonctionne correctement. Vos cas de test doivent notamment permettre de contrôler : le changement de vitesse et de position pour chaque paroi (une ou plusieurs).